

Introduction to MATLAB and modeling in IRIS

February 19, 2019



MATLAB basics

MATLAB basics

Desktop

Variables, commands

Functions

Scripts

Search Path

Help



Desktop

- MATLAB desktop displays four windows
 - **Command Window** is where commands are typed (**Ctrl+0**)
 - **Command History** is the list of earlier commands (**Ctrl+1**)
 - **Current Folder** is a simple file manager (deleting, copying, renaming etc.) (**Ctrl+2**)
 - **Workspace** is where the list of variables created earlier can be seen (**Ctrl+3**)
 - **Editor** is a text editor with many useful features (**Ctrl+4**)
- Windows can be replaced by dragging the headline (drag-and-drop), and be docked and undocked using the arrow next to the closing **X**
- Switching between windows is possible using shortcuts as well (see above)
- Many things can be set in the **File/Preferences** menu

MATLAB basics

Desktop

Variables, commands

Functions

Scripts

Search Path

Help



Variables, commands

Variables can be created using the `=`, for example `a = 2`

- Variable name is on the left-hand side, where
 - only letters (English ABC), numbers and the `_` character can be used
 - first character must be a letter
 - MATLAB is case-sensitive (upper- and lowercase letters are different)
- If we do not specify a variable name, the outcome will be automatically stored under the name `ans`
- Constant, operation, function, etc. can be on the right-hand side
- If a command is ended with `;`, then the outcome will not be displayed
- More than one command can be written in a line, separated by commas
- We can navigate between earlier commands using the `↑` and `↓` keys
- `clear`: deleting variables; `clc`: emptying the **Command Window**

MATLAB basics

Desktop

Variables, commands

Functions

Scripts

Search Path

Help



Functions

Example: `a = sqrt(2)`

- On the left-hand side the name of the variable containing the outcome
- On the right-hand side the name of the function, then the input between brackets
- In case of multiple inputs, we separate them using commas, for example `nthroot(2,3)` is resulting $\sqrt[3]{2}$
- When using multiple inputs, it may happen that functions behave differently depending on the number of inputs
- Functions can have multiple outputs
 - `[nr,nc] = size(A)` is the number of rows and columns of matrix `A`, respectively

MATLAB basics

Desktop

Variables, commands

Functions

Scripts

Search Path

Help



Scripts

Multiple commands can be collected in a file and run collectively

- As the file extension is .m, we call them M-files
- In an M-file, we can write a (one line) comment after the % character
- To run
 - type the name of the M-file into the **Command Window** (without extension)
 - open the M-file in the **Editor**, then press **F5**
 - if we intend to run only a particular part of the script, select it and press **F9**
 - a script can be divided to so-called cells using the %% characters; a cell can be run using the **Ctrl+Enter** shortcut

MATLAB basics

Desktop

Variables, commands

Functions

Scripts

Search Path

Help



Search Path

To run an M-file, MATLAB needs to know where the file is located

- If there is an M-file in the **Current Folder** under the given name, then it will run
- If not, MATLAB will look for the file in the directories stored in **Search Path**
- **Search Path** is a list of directories where we store M-files
- Order matters! MATLAB will run the first match
- **Search Path** can be set at **File/Set path**

MATLAB basics

Desktop

Variables, commands

Functions

Scripts

Search Path

Help



Help

There are two ways to get help on the behaviour of a function/command

- Type `help <name of command>` into the [Command Window](#), for example `help sqrt`; then a short description will appear in [Command Window](#)
- Using the command `doc` we can obtain a more detailed description in [Help Browser](#)
- We can search any expression in [Help Browser](#), not only the name of a command or function



Basic operation

Basic operation

Matrices

Logical variables

Basic operations

Statistical functions

Linear algebra, polynomials



Defining matrices

- Typing `A = [1,2;3,4]` will create matrix $\begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix}$
- Elements within a row are separated by `,` while rows by `;`
- Another possibility is to use `Space` instead of comma and `Enter` instead of semicolon
- `Space` usually does not matter within a command. There are exceptions though, for example
 - `[1 -1]` returns vector $\begin{pmatrix} 1 & -1 \end{pmatrix}$
 - while `[1 - 1]` returns constant 0
- From MATLAB's point of view, all variable are matrices (a constant is a 1x1 matrix), so no special syntax is needed

Special matrices and vectors I

- $A = \text{zeros}(N,M)$ returns an $N \times M$ matrix containing only 0s
- $A = \text{ones}(N,M)$ returns an $N \times M$ matrix containing only 1s
- $A = \text{eye}(N)$ returns an $N \times N$ identity matrix
- $A = 1:5$ equals $A = [1 \ 2 \ 3 \ 4 \ 5]$
- $A = 1:2:6$ equals $A = [1 \ 3 \ 5]$
- $A = a:b:c$ returns a row vector, where
 - the first element is a
 - the distance of two neighbouring element is b
 - the last element is just smaller, than c

a, b and c can be any real number. In case b would be negative, modify the above accordingly

Special matrices and vectors II

- `A = linspace(a,b,n)` returns a row vector with n element, where
 - the first element is a
 - the last element is b
 - the distance of any two neighbouring elements is identical
- Operations that make no sense (for example `0/0`) result NaN (Not-a-Number). The command `A = nan(N,M)` returns an $N \times M$ matrix containing only NaN
- The command `1/0` returns Inf. The command `A = inf(N,M)` returns an $N \times M$ matrix containing only Inf
- The command `A = rand(N,M)` returns an $N \times M$ matrix whose elements are random draws from the uniform distribution on $[0, 1]$
- The command `A = randn(N,M)` returns an $N \times M$ matrix whose elements are random draws from the standard normal distribution

Matrix transformation I

- `C = [A, B]` horizontally concatenates A and B
- `C = [A; B]` vertically concatenates A and B
- `D = diag(V)` returns
 - a diagonal matrix in which the main diagonal contains the elements of V , in case V is a vector
 - a vector containing the main diagonal of V , in case V is a matrix
- `C = blkdiag(A,B)` defines a block-diagonal matrix from the inputs (more than two inputs is possible)
- `B = tril(A)` returns the lower triangular part of A (with 0s in the upper part)
- `B = triu(A)` returns the upper triangular part of A (with 0s in the lower part)

Matrix transformation II

- $B = A'$ is the transpose of A
- $B = \text{flipud}(A)$ flips A horizontally, that is, its first row becomes the last, the second becomes the penultimate etc.
- $B = \text{fliplr}(A)$ flips A vertically
- $B = \text{rot90}(A)$ rotates A by 90 degrees, anticlockwise

Matrix transformation III

- $B = \text{repmat}(A,N,M)$ (or $B = \text{repmat}(A,[N \ M])$) returns an $N \times M$ tiling of A (try it!)
- $B = \text{reshape}(A,N,M)$ (or $B = \text{reshape}(A,[N \ M])$) reshapes matrix A
 - Elements of A are taken one-by-one, columnwise
 - B are filled the same way, columnwise

Take for example $A = \begin{pmatrix} 1 & 2 & 3 & 4 \\ 5 & 6 & 7 & 8 \\ 9 & 10 & 11 & 12 \end{pmatrix}$ and type $B =$

$\text{reshape}(A,4,3)$. Then $B = \begin{pmatrix} 1 & 6 & 11 \\ 5 & 10 & 4 \\ 9 & 3 & 8 \\ 2 & 7 & 12 \end{pmatrix}$.

Matrix indexing I

- $A(2,3)$ is the third element of the second row of matrix A
- $A(11)$ is the 11th element of A , proceeding columnwise
- $A(2,:)$ is the second row of A
- $A(:,3)$ is the third column of A
- $B = A([2 \ 4],[3 \ 1])$ is defined by the third and first element of the

second and fourth row of A . If $A = \begin{pmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \\ 10 & 11 & 12 \end{pmatrix}$, then $B = \begin{pmatrix} 6 & 4 \\ 12 & 10 \end{pmatrix}$.

- To sum up, we can index rows and columns using vectors whose elements are positive integers
 - 0 cannot be an index
 - it is the order of elements that matters in an indexing vector

Matrix indexing II

- `A(1,end)` is the last element of the first row of A
- `A(end-1,2)` is the penultimate element of the second column of A
- `A(1,1:end-1)` is the first row of A , except for its last element
- `A(end-1:-1:1,2)` is the second column of A in reverse order, except for its last element
- `A([1 end],:)` is the first and last row of A
- In sum: when indexing, command `end` denotes the size of the appropriate dimension of the matrix (i.e. number of rows or columns)
- `length(v)` is the length of vector v ; if we type `s = size(A)` then `s(1)` is the number of rows, `s(2)` is the number of columns of matrix A
- `[r,c] = size(A)` gives `r` as the number of rows and `c` as the number of columns of matrix A

Matrix indexing III

- Analogous to the above, we can replace elements of a matrix
- $A(2,3) = 5$ replaces the second element of the third row of A with 5
- If there is a matrix on the left-hand side, then the right-hand side must be of equal size
- Unless
 - right-hand side is constant
 - RHS and LHS are vectors of the same length (e.g. we can replace the elements of a column with the elements of a row vector)

Matrix indexing VI

- Another way of referring to the elements of a matrix is *serial indexing*
- The elements of the matrix are ordered, first down the columns and then by the columns
- By serial indexing we refer to this order
- If

$$A = \begin{pmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{pmatrix}$$

then $A([2 \ 5 \ 3 \ 6])$ is $[4 \ 5 \ 7 \ 9]$

- If the index vector is a column vector, then the result will also be a column vector
- Important special case: $A(:)$ is the column vector where the columns of A are stacked on top of each other

Basic operation

Matrices

Logical variables

Basic operations

Statistical functions

Linear algebra, polynomials



Logical variables

- So far we have only been dealing with numeric variables (NaN and Inf are special numeric values)
- Logical variables can take on two values only: `true` (displayed as `1`) or `false` (displayed as `0`)
- Logical variables can be arranged to matrices
- `true(N,M)` returns an $N \times M$ in which every element is `true`
- `false(N,M)` returns an $N \times M$ in which every element is `false`

Relational and logical operations

- Logical variables are typically created by using relational operators ($<$, $<=$, $>$, $>=$, $==$, $\sim$)
- These function the same way as basic operators
- For example, command $I = A < B$ results logical matrix I that is equal in size with matrices A and B , in which an element is **true** if the corresponding element of A is less than the corresponding element of B
- If we type command $K = I \& J$, an element of K will be **true** if the corresponding elements of I and J are **true**
- If we type command $K = I \mid J$, an element of K will be **true** if at least one of the corresponding elements of I and J is **true**
- If instead of the above we type $\&\&$ or $\mid\mid$, then the second operator will only be evaluated in case the first does not specify clearly the result
- Note that this only works for scalars

Logical indexing

- Logical matrices are frequently used for indexing
- If A is a numeric matrix and I is a logical matrix of equal size, then $A(I)$ consists of those elements of matrix A where the value of I is **true**
- Logical and numeric indexing can be mixed
- If A is a 4×3 matrix then $A(1,[\text{true false true}])$ results the first and third elements of the first row
- The most common use: $A(\text{abs}(A)<1)$ consists of the elements of matrix A whose absolute value is less than 1

Basic operation

Matrices

Logical variables

Basic operations

Statistical functions

Linear algebra, polynomials



Basic operations

- Golden rule: basic operations are referring to matrices, i.e.
 - inputs must be of appropriate size
 - or one of the inputs should be a scalar
- $A+B$ and $A-B$ requires A and B to be of equal size
- $A*B$ requires the number of columns of A and rows of B to be equal
- A^n requires A to be square matrix
- $\text{inv}(A)$ is the inverse of square matrix A
- A/B is the same as $A*\text{inv}(B)$, only it is faster and more exact
- $B\backslash A$ is the same as $\text{inv}(B)*A$, only it is faster and more exact
- Multiplication, exponentiation and division can be performed element-wise as well, using the commands $.*$, $.^$, $.\backslash$

Elementary functions

- Without being exhaustive: `abs`, `exp`, `log`, `log2`, `log10`, `sqrt`, `sin`, `cos`, `tan`, etc.
- Common feature is being *vectorized*
- That is, if x is a matrix and $y = \log(x)$, then y is a matrix too, where each element is the logarithm of the corresponding element of x
- `log(x)` and `sqrt(x)` returns complex values for negative x s, while `reallog(x)` and `realsqrt(x)` returns an error message

Arithmetic functions

- `rem(a,b)` remainder after division, `a,b` can be non-integers as well
- `round(a)`, `fix(a)`, `floor(a)`, `ceil(a)` rounding (standard, towards 0, up, down)
- `factor(n)` prime factors
- `primes(n)` prime numbers less than or equal to `n`
- `factorial(n)` $n!$
- `nchoosek(n,k)` if `n` is a scalar then $\binom{n}{k}$; if `v` is a vector then `nchoosek(v,n)`, all possible combinations
- `perms(v)` permutations of the elements of vector `v`
- `randi(m,n,k)` an $n \times k$ random matrix, every element is uniform on integers $1, 2, \dots, m$
- `randperm(m)` returns a random permutation of integers $1, 2, \dots, m$

Basic operation

Matrices

Logical variables

Basic operations

Statistical functions

Linear algebra, polynomials



Statistical functions I

- Common feature: input is usually a data matrix, where rows are for observations and columns are for variables
- The most important ones: `sum`, `prod`, `mean`, `median`, `var`, `std`, `min`, `max`, `diff`
- For example, if X is a $T \times n$ matrix, then `sum(X)` is a row vector of length n , where the i th element is the sum of the elements in the i th column of X ; all the above functions work in an analogous way
- The sum of rows is given by the command `sum(X,2)`
- Exceptions: `min`, `max` and `diff`
 - `min(X,2)` compares the elements of X to 2. Rowwise minimum is given by command `min(X,[],2)`
 - `diff(X,2)` calculates the second difference columnwise. Rowwise difference is given by command `diff(X,[],2)`

Statistical functions II

- $S = \text{cumsum}(X)$ calculates the cumulative sum for each column of X .
 S will be the same size as X
- $P = \text{cumprod}(X)$ returns the cumulative product
- $S = \text{sort}(X)$ sorts the elements of each column in ascending order (so that rows will not remain the same)
- $[S, I] = \text{sort}(X)$ also returns a collection of index vectors, specifying the order
- $S = \text{sortrows}(X, n)$ sorts elements based on the n th column, keeping rows unchanged
- $[S, I] = \text{sortrows}(X, n)$ also returns a collection of index vectors, specifying the order

Statistical functions III

- $C = \text{cov}(X)$ gives the covariance matrix of the columns of X
- $C = \text{corrcoef}(X)$ gives the correlation matrix of the columns of X
- $[C,P] = \text{corrcoef}(X)$ also returns P , a matrix of p -values for testing the hypothesis of no correlation.

Basic operation

Matrices

Logical variables

Basic operations

Statistical functions

Linear algebra, polynomials



Linear algebra I

- `inv(A)` is the inverse of square matrix A
- To solve a system of linear equations $Ax = b$, one should use the command `A\b` instead of `inv(A)*b`
- Advantages:
 - Faster, more exact
 - Works even if A is not a square matrix (type `help slash` to find out what happens in such cases)
- `null(A)` results an orthonormal basis for the null space of A , that is each element of `A*null(A)` is 0
- `pinv(A)` is the generalised inverse. For example, if $A = \begin{pmatrix} A & 0_{n,k} \\ 0_{k,n} & 0_{k,k} \end{pmatrix}$,

$$\text{then } \text{pinv}(A) = \begin{pmatrix} A^{-1} & 0_{n,k} \\ 0_{k,n} & 0_{k,k} \end{pmatrix}$$

Linear algebra II

- `det(A)` returns the determinant of matrix A
- `eig(A)` returns the eigenvalues of A
- `[V,D] = eig(A)` returns eigenvectors in V and eigenvalues in the main diagonal of D , so that $AV = VD$
- If S is a symmetric positive definite, then $P = \text{chol}(S)$ is the upper Cholesky decomposition: P is an upper triangular matrix and $S = P'P$
- `P = chol(S, 'lower')` returns the lower Cholesky decomposition: P is a lower triangular matrix and $S = PP'$
- `[Q,R] = qr(A)` returns the QR decomposition: Q is orthogonal ($QQ' = I$), R is upper triangular, and $A = QR$.

Polynomials

- In MATLAB, polynomials are defined by the vector of coefficients: vector $p = (p_1, p_2, \dots, p_{n+1})$ defines polynomial $p_1x^n + p_2x^{n-1} + \dots + p_nx + p_{n+1}$
- **polyval(p,x)** returns the values of polynomial p at x ; x can be a matrix as well
- **conv(p,q)** multiplies polynomials p and q
- **roots(p)** returns the roots of polynomial p
- **poly(r)** returns a polynomial, whose roots are the elements of vector r (main coefficient is 1)
- **polyfit(x,y,n)** fits a polynomial of degree n on observations (x_i, y_i)



Writing functions

Writing functions

Function vs. script

Variable types

Flow control

Setting default values

Efficiency



Example: OLS estimation I

- We would like to estimate the following :

$$y_t = c + \beta_1 X_t^1 + \beta_2 X_t^2 + \dots + \beta_n X_t^n + \varepsilon_t, \quad \varepsilon_t \sim NID(0, \sigma^2)$$

- This can be expressed as:

$$y_t = [1, X_t^1, X_t^2, \dots, X_t^n] \begin{bmatrix} c \\ \beta_1 \\ \beta_2 \\ \vdots \\ \beta_n \end{bmatrix} + \varepsilon_t$$

- If we have T observations, then

$$Y = X\beta + e$$

where Y and e are column vectors of length T , X is a $T \times n$ matrix and each row corresponds to one observation.

Example: OLS estimation II

- We know that the OLS estimation of β and σ^2 can be obtained as follows:

$$\hat{\beta} = (X'X)^{-1} X'Y$$
$$\hat{\sigma}^2 = \frac{1}{T-n} \sum_{t=1}^T (y_t - X_t \hat{\beta})^2$$

- Therefore, the code `ols.m` performing the OLS estimation could be for example:

```
[T,n] = size(X);  
X = [ones(T,1) X];  
n = n+1;  
bhat = (X'*X)\(X'*Y);  
resid = Y-X*bhat;  
s2hat = sum(resid.^2)/(T-n);  
c = bhat(1);  
bhat = bhat(2:end);
```

Example: OLS estimation III

- However, this solution is inconvenient from several aspects:
 - Inputs must be renamed each time to **X** and **Y**
 - The name of outputs will always be **bhat**, **c** and **s2hat**
 - **T** and **n** are auxiliary variables needed only for calculation, mostly we do not need them as outputs
- The above can be a problem especially when we would like to perform more estimation in a row, for example due to a Chow test, time-varying coefficients, etc.

Example: OLS estimation IV

- Solution: let's transform our script to a function. For this, we need to insert a row:

```
function [bhat,c,s2hat] = ols(Y,X)
```

```
[T,n] = size(X);  
X = [ones(T,1) X];  
n = n+1;  
bhat = (X'*X)\(X'*Y);  
resid = Y-X*bhat;  
s2hat = sum(resid.^2)/(T-n);  
c = bhat(1);  
bhat = bhat(2:end);
```

- Thus, estimating a simple consumption function looks as follows:

```
[inc_el,aut_c,errvar] = ols(log(cons),log(inc))
```

assuming that variables `cons` and `inc` exist.

Basic properties of functions

- It is the filename that matters, functions are called in by typing it
- Each function has its own *local workspace*
 - Only inputs and variables created inside the function can be used
 - Only outputs will be transferred to the base workspace
 - It is allowed to use variable names that are identic to those in the base workspace
- Inputs and outputs are defined by their order
- A multiple input function can be called with less than all inputs as well
- However, each used input has to be specified

Writing functions

Function vs. script

Variable types

Flow control

Setting default values

Efficiency



Structures I

- We expect other outputs from an OLS estimation as well: residuums, covariance matrix of coefficients, R^2 , p -values, t -statistics, F -statistics, etc.
- In theory, those can be given as separate outputs. However, there is a better solution: structures
- The main advantage of using structures is the ability to summarize multiple things in one variable, to which we can refer after by names
- Let's see an example

Structures II

```
function res = ols(Y,X)

[T,n] = size(X);
X = [ones(T,1) X];
n = n+1;
bhat = (X'*X)\(X'*Y);
resid = Y-X*bhat;
s2hat = sum(resid.^2)/(T-n);

res.const = bhat(1);
res.beta = bhat(2:end);
res.ser = sqrt(s2hat);
res.resid = resid;
res.R2 = 1 - sum(resid.^2)/sum((Y-mean(Y)).^2);
res.covmat = s2hat*inv(X'*X);
res.stderr = sqrt(diag(res.covmat));
res.tstat = bhat./res.stderr;
res.pval = 2*(1 - tcdf(abs(res.tstat),T-n));
res.Fstat = res.R2/(n-1)/((1-res.R2)/(T-n));
res.Fpval = 1 - fcdf(res.Fstat,n-1,T-n);
res.llh = -T/2*(1 + log(2*pi) + log(sum(resid.^2)/T));
res.bic = -2*res.llh/T + n*log(T)/T;
```

Structures III

- Structure `res` has *fields* that can be referred to by their names, e.g. `plot(res.resid)`
- `rmfield(res, 'R2')` removes field `R2`
- `fieldnames(res)` returns the names of the fields in a structure
- `isfield(res, 'R2')` is true if `res` has a field named `R2`
- Structures with the same number of fields can be arranged to arrays ('matrices'), for example `res = [res1 res2]`. Then `res(1)` returns the same as `res1`
- Each `R2` fields of a structure array `res` can be referred to as `[res(:).R2]` (more on this later)
- *Dynamic field names*: if `fn = 'R2'` is a character variable containing the name of a field, then `res.(fn)` is the same as `res.R2`

Character type variables

- `s = 'a'`
- Can be arranged to arrays as well: in case of `s = 'A sentence.'`, `s` will be a 1×11 character array (vector, string)
- Indexing works the same way as we have seen so far: using the above example, `s(1:5)` returns `'A sen'`
- Concatenation is the same, too: `['A' ' sentence.']}`
- `strvcat(S1,S2)` vertically concatenates two strings (can be of different length)
- `strcmp(S1,S2)` compares two strings (can be of different length)
- Additional functions: `lower`, `upper`, `strcmpi`, `strfind`, `strrep`, `regexp`

Cell arrays

- Arranging different type variables into an array
- `c = {a,b;c,d}`. The point is that `a,b,c,d` can be variables of any type or size
- Mostly we use it to store strings: `c = {s1,s2,s3,s4}`
- Calculation is not possible with cells, but we can transpose, rotate and reshape them
- Indexing
 - `c{i,j}` returns the content of the *j*th cell of row *i*
 - `c(i,j)` returns the *j*th cell of row *i*
- Concatenation
 - `[c1,c2]` „takes“ the contents of cells and concatenates them into a new cell
 - `{c1,c2}` concatenates the cells themselves, „cell in cell“

Comma-separated lists

- Comma-separated lists
- We obtain it when
 - `c` is a cell and we type `c{:}`
 - `s` is a structure array and we type `s(:).fn`
- If concatenation can be interpreted on the elements of the list, then command `[S(:).R2]` will make sense as well
- Also, any function works where list elements would be reasonable arguments

Transforming variable types

- `cellstr(S)` creates cell array from character array `S`
- `strvcat(C)` the reverse operation
- `num2str(A,N)` converts numeric array `A` into a character array, `N` is the number of decimal places (default is about 4)
- `num2cell(A)` converts array `A` into the cells of a cell array

Writing functions

Function vs. script

Variable types

Flow control

Setting default values

Efficiency



For loop

- Example:

```
s = 0;  
N = length(v);  
for i = 1:N  
    s = s + v(i);  
end
```

- The running index (here **i**) goes through the columns of the matrix following the equals sign (here row vector **1:N**), that is if **v** is a row vector, we can type the following instead of the above:

```
s = 0;  
for vi = v  
    s = s + vi;  
end
```

- **v** can be of any variable type

While loop

- Example:

```
prevs = -1;  
s = 0;  
i = 1;  
while prevs < s  
    prevs = s;  
    s = s + 1/i^2;  
    i = i+1;  
end
```

- Question: what does this program do?

Conditional statement: if

- Example:

```
a = -2;  
if a > 0  
    b = log(a);  
elseif a < 0  
    b = log(-a);  
else  
    b = nan;  
end
```

- If the condition is not a scalar, the **if**-block runs only when each element is **true**
- **isequal(A,B)** returns a scalar, it can compare variables of any type and size

Conditional statement: switch

- Example:

```
day = 'T';  
switch day  
    case {'M' 'T' 'W' 'Th' 'F'}  
        disp('Working day')  
    case {'Sa' 'Su'}  
        disp('Weekend')  
    otherwise  
        disp('Something went wrong.')  
end
```

- A more legible version of the `if`-version
 - Only applicable when each condition is equality
 - Works with strings as well
 - Condition can be multi-element

Interrupting program execution

- **return** quits the function
- **break** quits the loop
- **continue** skips the remaining part in the current iteration of the loop and continues with the next iteration
- **pause(t)** halts execution for **t** seconds
- **error(msg)** aborts function and displays **msg** error message
- **warning(msg)** displays **msg** warning message

Ignore errors

- Example:

```
v = randn(1,2);  
for vi = v  
    try  
        a = reallog(vi);  
    catch  
        warning('Something went wrong')  
    end  
end
```

- Not recommended, we should correct the code instead

Writing functions

Function vs. script

Variable types

Flow control

Setting default values

Efficiency



Default values I

- Let's examine the following modification of `ols.m`:

```
function [bhat,c,s2hat] = ols(Y,X,ifconst)

[T,n] = size(X);

if ifconst
    X = [ones(T,1) X];
    n = n+1;
end

bhat = (X'*X)\(X'*Y);
...
```

- This way, we can specify whether we want a default value in the function
- However, we almost always want it. How can we set a default value for input `ifconst`?

Default values II

- The function `nargin` returns the number of input arguments passed in the call to the currently executing function
- Setting default:

```
function [bhat,c,s2hat] = ols(Y,X,ifconst)
```

```
if nargin < 3  
    ifconst = true;  
end
```

```
[T,n] = size(X);  
...
```

- The function `nargout` returns the number of input arguments passed in the call to the currently executing function. It is useful as well, sometimes.

Default values III

- We refine further:

```
function [bhat,c,s2hat] = ols(Y,X,ifconst,iflog)

if nargin < 4
    iflog = false;
end

if nargin < 3
    ifconst = true;
end

if iflog
    Y = log(Y);
    X = log(X);
end

[T,n] = size(X);
...
```

Default values IV.

- If we do not want to call `iflog` with its default input, we need to specify four inputs when calling
- In such cases, `ifconst` will never get default
- Question: how can we set default value of the n th input, when we want to specify more than n inputs?
- Answer:
 - Specify a nonsensical value, in case of most MATLAB function this can be the empty matrix `[]`
 - If we give this for the n th input, then the function will use the default
 - For testing, use function `isempty`

Default values V.

- So:

```
function [bhat,c,s2hat] = ols(Y,X,ifconst,iflog)

if nargin < 4
    iflog = false;
end

if nargin < 3 || isempty(ifconst)
    ifconst = true;
end

if iflog
    ...
end
```

- Thereafter the command `ols(Y,X,[],1)` takes logarithm, and include a constant term in the equation by default
- All this is necessary because inputs are identified by their position

Writing functions

Function vs. script

Variable types

Flow control

Setting default values

Efficiency



tic-toc

- **tic** starts stopwatch timer (i.e. saves current time)
- **toc** stops stopwatch and displays time elapsed since **tic**. **t = toc** saves elapsed time into **t**
- Example:

```
N = 500;  
A = randn(N);  
x = randn(N,1);  
b = A*x;
```

```
tic  
x1 = inv(A)*b;  
toc
```

```
tic  
x2 = A\b;  
toc
```


Profiler

- A disadvantage of **tic-toc** is that in case of a longer block, it does not reveal which command slows down the execution
- Solution: **Profiler**
 - Can be found on the **Toolbar** near the **Help** button
 - Type the command to be tested into the upper field
 - We will see for each row how much time does it take when being executed
 - Moreover, this applies to all subfunction called in



Data and dates

Data and dates

Excel file format

Import Wizard

Reading data from Excel using functions

Handling dates

Saving and writing data



Reading data from Excel: format

- Excel file should be formatted as follows:

	Var1	Var2	Var3
jan.12	1	5	9
apr.12	2	6	10
jul.12	3	7	11
okt.12	4	8	12

- Names in the first row, dates in the first column. Or inversely. Either can be omitted as well.
- Date format is not important, either (but it should be a cell of 'Date' type)
- Only numeric values* should fill the fields containing values
- In case of missing data, the corresponding cell should be empty
- Differently formatted files can be handles as well, but then

Data and dates

Excel file format

Import Wizard

Reading data from Excel using functions

Handling dates

Saving and writing data



Reading data from Excel: copy-paste

- We select data in Excel then **Ctrl+C**
 - Only numeric values
 - Empty cells are not allowed
 - Decimal separator should be a point
- In the **Command Window**
 - Type e.g. **data = [**
 - Press **Ctrl+V**
 - Type **];**
- Thereby a matrix named **data** is created with the desired data



Reading data from Excel: Import Wizard I

- Open Excel in **Current Directory** (left-click on it twice, or right-click and **Open**)
- Choose Sheet in the left panel
- Data can be seen in the right panel
 - Numeric data on the **data** tab
 - String data in a cell array on the **textdata** tab
- After hitting **Next**, we can choose which output to create
- Hit **Finish**

Reading data from Excel: Import Wizard II

- If Date column is missing, a new tab named **colheaders** will appear in the right panel, where we can see variable names
- After hitting **Next**, there is a new option in the upper panel: **Create vectors from each column using column names.**
- By choosing it, it will happen
- If first row is missing, variables can be created from rows
- Handling dates and variable names together requires writing a new function

Data and dates

Excel file format

Import Wizard

Reading data from Excel using functions

Handling dates

Saving and writing data



Reading data from Excel: function

- `[num,txt,row] = xlsread(fname, sheet, range)`, for example
`[num,txt,row] = xlsread('data.xlsx',1,'a1:f100')`
- Inputs:
 - `fname` file name
 - `sheet` name or number of sheet. If value of `sheet` is `-1`, then Excel will be opened and we can choose data to be read. First sheet is the default value.
 - `range` see example above
- Outputs:
 - `num,txt` is the same displayed earlier on tabs `data` and `textdata`
 - `row` returns data read from Excel arranged into a cell array. To be used when numeric and string values are mixed in the data.

Function to read data from Excel

```
function [data,names,dates] = readxls(fname, sheet, range)

if nargin < 3
    range = '';
end

if nargin < 2
    sheet = '';
end

[data,txt] = xlsread(fname,sheet,range);
names = txt(1,2:end);
dates = datenum(txt(2:end,1),'yyyy.mm.dd');
```

Data and dates

Excel file format

Import Wizard

Reading data from Excel using functions

Handling dates

Saving and writing data



Handling dates

- Days are numbered, the first day is 0000.01.01. Fraction part indicates intraday time
- `datestr(D,F)` is the string format display of D th day. D can be a vector as well
- F describes the string format of the date. It can be between 0 and 31, or a string, e.g. `'yyyy.mm.dd'`. See `help datestr`
- `datevec(D,F)` returns a date vector: [year, month, day, hour, minute, second]
- `datenum(D,F)` number corresponding to days in D . D can be cell or character array (each row is a day)
- Related functions: `date`, `now`, `addtodate`, `eomday`, `weekday`

Data and dates

Excel file format

Import Wizard

Reading data from Excel using functions

Handling dates

Saving and writing data



Saving and writing data

- The command `save data` saves variables into a file named `data.mat`
- These can be loaded again using the command `load data`
- `xlswrite(fname,data,sheet,range)` works analogous to function `xlsread`
- Dates and variable names are to be handled in a separate function, again

Function to write data in Excel

```
function writexls(fname,data,names,dates)

if isnumeric(data)
    data = num2cell(data);
end

if nargin > 2 && ~isempty(names)
    data = [names;data];
end

if nargin > 3
    if isnumeric(dates)
        dates = cellstr(datestr(dates,'yyyy.mm.dd'));
    elseif ischar(dates)
        dates = cellstr(dates);
    end
    if ~isempty(names)
        dates = [{''];dates];
    end
    data = [dates,data];
end

xlswrite(fname,data);
```




Plotting

Plotting

- Plotting functions

- Simple customization

- Full customization

- 3D plots

- Other



plot

- `plot(x,y)` creates a scatter plot. `x` and `y` can be matrices as well
- `plot(x1,y1,x2,y2,...)` obvious
- `plot(x,y,':r')` draws with a dashed red line, using star marker. For further options, see `help plot`
- `plot(x1,y1,'spec1',x2,y2,'spec2',...)`
- Examples:
 - Horizontal line from `a` to `b` at the level of `c`: `plot([a b],[c c])`
 - Vertical line at `a`, from `b` to `c`: `plot([a a],[b c])`
 - `n`-sided regular polygon

```
n = 7;
a = linspace(0,2*pi,n+1)';
plot(cos(a),sin(a),'.-')
axis equal
```

Other settings

- How can we set other properties (line width, marker size, arbitrary color, etc.)

- `plot(x,y,'spec','propty1',val1,'propty2',val2,...)`

- Example:

```
plot(x,y,':sq',...  
      'linewidth',2,...  
      'color',[0.6 0 0],...  
      'markersize',12,...  
      'markerfacecolor',[0.4 0.8 0.4],...  
      'markeredgecolor',[0 0 0.7])
```

- Separate lines cannot be customized with this solution

bar, barh

- `bar(x,y)` creates a bar diagram, `y` can be a matrix
- `bar(x,y,0.7)` changes bar width (default is 0.8)
- `bar(x,y,'style')` makes sense especially when we have more data series. Possible values: `'grouped'`, `'stacked'`, `'hist'`, `'histc'`
- `bar(x,y,'color')`, e.g. `bar(x,y,'r')` sets the color of the bars
- Further options:

```
bar(x,y,0.7,...  
    'facecolor',[0.6 0 0],...  
    'edgecolor',[0.6 0.6 0.9],...  
    'linestyle',':',...  
    'linewidth',4)
```

- Separate bars cannot be customized, either

hist, histc

- `hist(x)` creates a histogram with 10 bins. `x` can be a matrix as well
- `hist(x,n)` creates a histogram with `n` bins
- `hist(x,v)` creates a histogram where intervals between the center of bins are determined by the elements of vector `v`
- `[n,v] = hist(...)` returns frequencies and centers of bins
- `[n,ind] = histc(x,v)` returns vector `v` containing the edges of bins. Without drawing a figure, `ind` contains which bins were the elements of `x` put into

area

- `area(x,y)` returns an area plot, cumulative, the area between given data series is colored
- `area(x,y,b)` sets `b` as a base value, to which the first data series is related. Default `b=0`
- Customisation

```
load fchrt
area(dates,data,...
     'facecolor','b',...
     'edgecolor','r',...
     'linewidth',2,...
     'linestyle','-.')
```

- Lack of customizability of separate data series is particularly annoying

pie, pie3

- `pie(x)` pie chart. Values of `x` must be positive
- `pie(x,expl)` offsets certain slices from the pie. `expl` is a logical vector of equal size than `x`, slices are offset when `expl` takes on values
- `pie(x,lbl)` or `pie(x,expl,lbl)` can overwrite default labels. `lbl` is a cell array
- `pie3` works the same way, creates a 3D pie chart

Right and left axis

- `plotyy(x1,y1,x2,y2)` displays `x1,y1` on the left, `x2,y2` on the right
- If we want a bar chart, use command `plotyy(x1,y1,x2,y2,@bar)`
- After testing, it is obvious that this makes little sense
- Good solution: later

Multiple figures in one axis

- The above commands create a new figure each time
- This can be changed using the command **hold**
- Example:

```
plot(x1,y1)  
hold on  
plot(x2,y2)  
hold off
```

- Various figure types (plot, bar, area) can be merged
- Command **hold all** prevents the resetting of coloring
- If we want more figures in separate windows, the command **figure** creates a new window

Multiple axes in one window

- Frequently, we want to see multiple figures at the same time
- That is what function `subplot` is for
- Example:

```
X = (1:100)';  
Y = 1 + 0.2*X + randn(100,1);  
eq = ols(Y,X);  
subplot(2,1,1)  
plot([Y eq.const + X*eq.beta(:)])  
subplot(2,1,2)  
bar(eq.resid)
```

Plotting

Plotting functions

Simple customization

Full customization

3D plots

Other



Default settings

- `xlim([xmin xmax]), ylim([ymin ymax])` the limits of axes
- `xlabel('x-axis label'), ylabel('y-axis label')`
- `grid on, grid off`
- `title('Figure title')`
- `legend 'Series 1' 'Series 2' ... 'location' 'NorthWest'`
- These refer always to the last figure(current axis)

Date label

- Suppose that we have vectors `x` and `dates`, where `dates` contains date numbers
- We would like to create a figure with dates as labels of the x-axis label
- Solution:

```
plot(dates,cpi)  
datetick('x','yyyy-QQ')
```

- Using this, limits and ticks of the x-axis will change. It will remain unchanged, however, if we use

```
datetick('x','yyyy-QQ','keepticks','keeplimits')
```

Plotting

Plotting functions

Simple customization

Full customization

3D plots

Other



Things we were unable to set so far

- Customizing elements of line-, bar- and area plot
- Font size, color, type
- Axis ticks, arbitrary tick labels (e.g. countries)
- Appropriate handling of two vertical axes



Elements, IDs

- Figures consists of various elements: figure window, axes, lines, bars etc.
- Elements have properties, these are what we want to set
- For that reason, each element has an ID (handle graphics), with which we can refer to the given element
- This is usually given as an output of the plotting command
- For example, `H = plot(X)` returns vector `H`, which has as many elements as `X` has columns, because this is how many lines are on the axis
- The ID of the current axis is given by function `gca`

Example: date label

- The problem is that if tick marks of the horizontal axis is automatically generated, they will be round numbers
- This does not make too much sense if we are handling a date axis
- Manual setting of tick marks:

```
plot(dates,cpi,'.-')  
xlim([dates(1) dates(end)])  
grid on  
set(gca,'xtick',dates(2:4:end))  
datetick('x','yyyy-QQ','keepticks','keeplimits')
```

Commands get and set

- Command `set`

```
set(H,'Property1',Value1,'Property2',Value2,...)
```

- Command `get`

```
x = get(H,'Property');
```

- Example (horizontal line at zero)

```
function zeroline
```

```
xl = get(gca,'xlim');  
hold on  
plot(xl,[0 0],'k')  
hold off
```

Plotting

- Plotting functions
- Simple customization
- Full customization
- 3D plots
- Other



plot3 with vector inputs

- `plot3(x,y,z)` 3D line plot
- Plots the points `(x(i),y(i),z(i))`
- Example (spiral)

```
a = linspace(0,6*pi,100);  
x = cos(a);  
y = sin(a);  
z = a;  
plot3(x,y,z,'.-')
```



plot3 with matrix inputs

- `plot3(X,Y,Z)`, inputs are matrices of equal size
- Plots the points `(X(i),Y(i),Z(i))`
- Connects points in columns with the same index
- Example

```
X = repmat((-5:5)',1,3);  
Y = repmat(1:3,11,1);  
Z = X.^2 + Y;  
plot3(X,Y,Z,'.-')  
grid on
```

meshgrid

- Generating matrices **X** and **Y** in the previous example was rather cumbersome
- This is the task function **ndgrid** was invented for
- Using it in the previous example:

```
[X1,Y1] = ndgrid(-5:5,1:3);  
Z1 = X1.^2 + Y1;  
plot3(X1,Y1,Z1,'.-')  
grid on
```

- Function **meshgrid** works similarly (inputs and outputs have to be interchanged)

```
[Y2,X2] = meshgrid(1:3,-5:5);  
Z2 = X2.^2 + Y2;  
plot3(X2,Y2,Z2,'.-')  
grid on
```

- `[X1,Y1] = ndgrid(-5:5)` returns the same as `[X1,Y1] = ndgrid(-5:5,-5:5)`

mesh, surf

- Similar to function `plot3`
- But all „matrix neighbour“ points are connected
- And also colored, depending on the value of `Z`
- `mesh` colors edges, `surf` colors faces
- Colors can be set using the command `colormap`
- Examples in the `figures.m` file



Plotting

- Plotting functions
- Simple customization
- Full customization
- 3D plots
- Other



Saving

- Copying to Word
 - Click on **Edit/Copy Figure** in the menu of the figure, then paste
 - Copying settings (background color, font size, etc.) can be found in the menu of **Command Window** under **File/Preferences/Figure Copy Templates** and **File/Preferences/Figure Copy Templates/Copy Options**
- Saving to file
 - Click on **File/Save as...** in the menu of the figure, then choose file format on the bottom
 - Settings can be found under **File/Export Setup...** in the menu of the figure
- By clicking on **File/Generate code...** in the menu of the figure, we can create a function whose execution will generate the figure later

Some useful things

- `axis equal` sets scale equal on axes, useful if the shape of the displayed figure matters (see regular polynom example earlier)
- `axis off` making axes disappear (rarely used)
- `axis vis3d` fixes scales, useful if we want to rotate a 3D figure
- `linkaxes([h1,h2])` connects axes with indices `h1` and `h2` (same limits, tick marks, moving together, etc)



IRIS basics

IRIS basics

Components of IRIS

Dates

Time series

Databases



Components of IRIS

- IRIS is a collection of MATLAB functions to facilitate dealing with economic time series and working with models
- Such an extension pack is called a *toolbox*
- The IRIS toolbox has four main components
 - Handling data: dates, time series, databases, named matrices
 - Working with economic models: forecasting, filtering unobserved variables, Bayesian estimation of parameters, tuning forecasts
 - Generating reports to present and share results: graphs and tables of model variables, model properties, etc.
 - Statistical time series models: VAR-s, structural VAR-s, Bayesian VAR-s, FA-VAR-s
- We will concentrate on the first two, and less on the last two

Setting up IRIS

- IRIS can be downloaded from <https://iristoolbox.codeplex.com>
- Installation of IRIS consists of three steps
 - unzip the downloaded .zip file into a folder
 - copy this folder to a suitable place on your computer (can be anywhere)
 - add the path of this folder to the MATLAB path
 - run `irisstartup` from the MATLAB command window
- The last two steps are usually contained in the `startup` file, using the `addpath` command
- After finished working with IRIS, run `irisfinish`
- IRIS is customizable; we will discuss the options when we get to the relevant topic

IRIS basics

Components of IRIS

Dates

Time series

Databases



IRIS date numbers

- Not the same as MATLAB or Excel date numbers
- IRIS dates are classified by frequency
- There are six possible frequencies, each having its own creator function

yearly	half-yearly	quarterly	bi-monthly	monthly	weekly
yy	hh	qq	bb	mm	ww

- Example: `qq(2016,1)` creates an IRIS date number representing the first quarter of 2016
- Use the `dat2str` function to convert IRIS date numbers into strings
- And `[y,p,f] = dat2yprf` is analogous to `datevec`: it returns the year, period, and frequency of an IRIS date number

Calculation with dates

- IRIS date numbers are defined so that adding 1 to a date number results in the date number of the next period, regardless of frequency
- `qqtoday + 1` is the IRIS date number of the next quarter
- Similarly, `qq(2016,1) - qq(2013,4)` gives 9, the number of periods between the two dates
- Caution! The command `qq(2016,1) - mm(2013,4)` will not throw an error, but the result is of course meaningless
- Use the `datdiff` function instead
 - `datdiff(qq(2016,1), qq(2013,4))` gives 9
 - `datdiff(qq(2016,1), mm(2013,4))` gives NaN

Creating ranges

- The `:` operator works for dates
- `qq(2001,1):qq(2016,1)` creates a date vector – or a *date range* – for each quarter between `qq(2001,1)` and `qq(2016,1)`
- But again, the `datrange` function is a better choice, since it checks the inputs
 - `datrange(qq(2001,1):qq(2016,1))` is fine
 - `datrange(qq(2001,1):mm(2016,1))` gives error
- See the IRIS Manual for other date functions, or type `help dates`

IRIS basics

Components of IRIS

Dates

Time series

Databases



tsseries objects

- In IRIS, time series are stored in **tsseries** objects
- The **tsseries** function creates **tsseries** objects
- The command **ts = tsseries(qq(2001,1),rand(20,1))**
 - creates the time series object **ts**
 - its first date is **qq(2001,1)**
 - its length is 20, so its last period is 2005Q4
 - and it is filled with random numbers

Using date ranges

- The first input to `tseries` can also be a date range
- But in this case, the second input must be a scalar, or the number of its rows must be the same as the number of periods in the range
- `tseries(qq(2001,1):qq(2016,1),1)` creates a time series that is constant 1
- `tseries(qq(2001,1):qq(2016,1),1:61)` creates a time trend
- But for the latter, there is an easier solution using the `dat2ttrend` function
`tseries(qq(2001,1),dat2ttrend(qq(2001,1):qq(2016,1)))`

Multivariate time series

- A single time series object can contain multiple time series
- Example: `tsseries(qq(2001,1),randn(20,3))` will create a time series object with 3 variables, all of them 20 periods long
- The number of variables in a time series is equal to the number of columns of the second input
- Another way to create multivariate time series is by horizontal concatenation

```
ts1 = tsseries(qq(2001,1),(1:10)');  
ts2 = tsseries(qq(2002,1),(1:10)');  
ts = [ts1 ts2];
```

- The range will be the union of the ranges of the inputs, the results will be padded with `NaN`-s

Functions

- The standard arithmetic operators (+, -, *, /, etc.) work as expected
 - The range of the operands can be different, but the frequency must be the same
 - The range of the result will be truncated, i.e. NaN-s are removed from the beginning and from the end
- The same is true for the logical operators (=, ~, <, >, etc.)
- Most elementary functions can be used with time series.

Getting properties

- The following functions return various useful properties of a time series

<code>startdate</code>	First observation date
<code>enddate</code>	Last observation date
<code>length</code>	Length of the series (difference of the previous two plus 1)
<code>freq</code>	Frequency of the series

- And these are useful for multivariate time series

<code>size(.,2)</code>	Number of variables in the time series
<code>get(.,'nanStart')</code>	First date in which all variables have observations
<code>get(.,'nanEnd')</code>	Last date in which all variables have observations

Referencing

- `ts(qq(2004,1))` returns the observations for 2004Q1
 - if `ts` is multivariate, then the result is a row vector
- `ts{qq(2004,1)}` returns a *time series object*, with the observations for 2004Q1 (note that curly brackets are used here)
- `ts{qq(2004,1):qq(2004,4)}` returns a time series object, with the observations between 2004Q1 and 2004Q4
 - it is enough to specify the start and end of a range:
`ts{[qq(2004,1) qq(2004,4)]}` gives the same result
 - we can use the `end` keyword as well: `ts{qq(2004,1):end}` returns all observations from 2004Q1
- `ts{qq(2004,1):qq(2004,4),1}` returns a time series object, with the observations *of the first variable* between 2004Q1 and 2004Q4

Assigning values

- The general form is the following
$$\text{ts}([\text{fd } \text{ld}], \text{var_ind}) = \text{values}$$
- Only brackets may be used
- **fd**, the first date in the assignment, can be earlier than the start date of **ts**
- **ld**, the last date in the assignment, can be later than the end date of **ts**
- **var_ind** selects the variables in a multivariate time series
- the matrix of the assigned value, **values**, must be of the same size as the selected elements on the left, or it can be a scalar
- In short, everything is as usual for numeric assignments

Concatenation

- Horizontal concatenation: see before
- `repmat(ts,1,3)` is the same as `[ts ts ts]`
- Vertical concatenation: `ts3 = [ts1; ts2]`
 - the start date of `ts3` is the earliest of the start dates of `ts1` and `ts2`
 - the end date of `ts3` is the latest of the end dates of `ts1` and `ts2`
 - for a date `d` in between, the value in `ts3` is the corresponding value in the last of the inputs
 - if `ts2` has a value for this date, then this will be the value in `ts3`
 - if not, but `ts1` has a value for this date, then this will be the value in `ts3`
 - otherwise it will be `NaN`

Conversion

- The `convert` function converts time series to different frequencies
- Some examples
 - `ts_q = convert(ts_m,'q')` converts a monthly time series `ts_m` into a quarterly series `ts_q`, by taking the quarterly averages
 - `ts_q = convert(ts_m,'q','method','first')` converts a monthly time series `ts_m` into a quarterly series `ts_q`, by taking the first month in each quarter
 - `ts_m = convert(ts_q,'m')` converts a quarterly time series `ts_q` into a monthly series `ts_m`, using cubic interpolation
- There are other options; type `help tseries/convert`

Filtering, x12

-



IRIS basics

Components of IRIS

Dates

Time series

Databases



Database in IRIS

- A database in IRIS is a MATLAB structure variable, some of whose fields can be time series objects
- The time series can have different frequencies
- Not all fields have to be time series
 - for example, we can collect model data and model parameters in a single database; the latter will be numeric (double) fields
- The easiest way to create a database is by direct assignment:
`db.ts1 = ts1`

Loading/saving data

- But most often we load the data from an external file with the `dbload` command
- It is best practice to store data in CSV files
 - first row contains the names of the variables
 - first column contains dates in standard IRIS format: `2001Y01`, `2000Q01`, `2000M01`
- In this case, simply type `db = dbload('filename.csv')`
- Other date formats can also be handled:
`db = dbload('filename.csv','dateformat','yyyy-mm-dd')`
 - but in this special case the frequency will be monthly, so we will have to convert it
- `dbsave('filename.csv',db)` will save the database into a CSV file
 - only the time series objects in `db` will be saved
 - all time series objects in `db` must have the same frequency

Batch processing data

- We may want to apply a function to each fields of a database
 - in the previous slide, we had to convert all monthly time series into yearly
- The easiest way to do this is with the function `dbfun`
- For the example above:
`dy1 = dbfun(@(x) convert(x,'a','method','first'),dy)`
- This will do the following
 - go through all fields in `dy`
 - substitute these fields in place of `x` into the function
`convert(x,'a','method','first')`
 - put the result into a field in `dy1` with the same name as in the input database
- Of course any other time series functions can be used

Combining databases

- `db3 = dbmerge(db1,db2)` will combine the the databases `db1` and `db2`, by overwriting fields from `db1` with those from `db2`
- `db3 = dboverlay(db1,db2)` will combine the databases `db1` and `db2`, by overwriting *observations* from `db1` with those from `db2`
- The difference is important when there are time series with the *same name*
 - if they also have the *same frequency*, then
 - `dbmerge` will use only the second time series
 - `dboverlay` will combine the observations, and overwrites the values in the first time series with those in the second for common dates
 - if the frequency is different, then
 - `dbmerge` will use only the second time series (same as above)
 - `dboverlay` will use only the first time series (tries to overwrite it, but cannot, since frequencies are different)

Further database operations

- `db2 = dbclip(db1, rng)` will clip all time series in `db1` to the range `rng`, provided that the frequencies match
- `[db2, db] = dbsplit(db, oldR, newR)` will take the fields from `db` whose names match `oldR`, and put them into the database `db2` with names given by `newR`. For example:

```
db.cpi_all = tseries(qq(2000,1),@rand);  
db.cpi_core = tseries(qq(2001,1),@rand);  
db.gdp      = tseries(qq(2002,1),@rand);
```

```
[cpi, db] = dbsplit(db, {'cpi_.*'}, 'infl_$1');
```

will result in

```
cpi =  
    infl_all: [1x1 tseries]  
    infl_core: [1x1 tseries]
```

```
db =  
    gdp: [1x1 tseries]
```



Modeling in IRIS

Modeling in IRIS

Models and their applications

Defining models

Filtering

Simulating/forecasting

Nonlinear models



Technical properties of models

- From a technical point of view, the following classification is useful to understand how IRIS works with models
 - Linear vs. nonlinear models
 - Forward-looking vs. backward looking models
 - All model variables are observed vs. there are unobserved variables
- The relevance of these properties is different for different tasks
 - For *filtering* unobserved variables, linearity is essential
 - Linearity is also useful if the model is forward-looking, but it is not essential
 - For *simulation* and for *forecasting* with backward-looking models, linearity is not an issue

Linear models

- Since filtering is an essential part of IRIS, *models will always be linearized*
 - Linearization is of course around the steady-state
- However, for simulation, we may instruct IRIS to use the non-linear version of an equation
 - But this rarely makes a substantial difference, since the linearized version is usually a good approximation around the steady-state
- We will therefore focus on linear models, with a simple nonlinear example at the end of the course

Steps of modeling

- We define the model in a separate file
- We use the **model** function to read the model definition, and to create the model object variable in MATLAB
- We **assign** to the model a database containing parameter values
- We then have to **solve** – i.e. linearize – the model
- We may then **filter** the unobserved variables
- And then we can **simulate** the model, or **jforecast** with it

Modeling in IRIS

Models and their applications

Defining models

Filtering

Simulating/forecasting

Nonlinear models



Model components

Defining the model means the following

- Defining the endogenous model variables
- Defining which of these are observed
- Defining the shocks in the model
- Defining the parameters
- Defining equations for the endogenous variables
- Defining how observations are related to endogenous variables
- Possibly defining observation errors (if there are any, in economic models usually there aren't)

IRIS keywords for the components

In the model definition file, we use special keywords to indicate which component of the model we are defining

Defining the endogenous model variables

!transition_variables

Defining which of these are observed

!measurement_variables

Defining the shocks in the model

!transition_shocks

Defining the parameters

!parameters

Defining equations for the endogenous variables

!transition_equations

Defining how observations are related to endogenous variables

!measurement_equations

Defining observation errors

!measurement_shocks

There are other keywords, but these are the most important ones

General rules for name declarations

- All the names must be unique, and each name must be declared exactly once
- We can add labels to a name as a MATLAB string before the name
- Names can be separated by commas, semicolon-s, or a line break
- Equations must be terminated by a semicolon
- Each keyword starts a *declaration block*
- There can more than one declaration block of the same type

Variable and parameters names

- In the **!transition_variables** block(s) we declare the names of the endogenous variables of the model. Some of these are observed, but there can be unobserved endogenous variables
- In the **!measurement_variables** block(s) we declare the names of the observed (measured) variables
 - Since names must be unique (see next slide), we can not have the same name for an observed endogenous variable and for its observations
 - It is advisable to use a general rule for the names of the observed variables: for example **gdp** might be the name of GDP in the model, and **gdp_obs** might be the name for the observations
- In the **!parameters** block(s) we declare the names of the model parameters

Shock names

- In the **!transition_shocks** block(s) we declare the names of the shocks/error terms/residuals of the endogenous model equations
- In the **!measurement_shocks** block(s) we declare the names of the measurement errors
- In standard economic models this block is rarely needed

Transition equations

- The equations of the model are given here
- A transition equation can contain
 - leads and lags of the transition variables
 - transition shocks, without leads or lags
 - parameters



Measurement equations

- The equations relating the observed variables to model variables
- Most often a measurement equation is a simple identity, like
`gdp_obs = gdp`
- A measurement equation can contain
 - measurement variables, no leads or lags
 - transition variables, no leads
 - measurement shocks, without leads or lags
 - parameters

Parameter values

- Parameter values must be provided as fields of a database (MATLAB structure)
 - If we defined **a1** and **a2** as our parameter names, then we must give their values as follows

```
pars.a1 = 0.5;  
pars.a2 = -0.7;
```
- *We must also give the standard deviations of the shocks*
 - If we defined **shock_gdp** as a shock (either transitional or measurement), then we must give its standard deviation in the same database
 - The name of the standard deviation is **std_** followed by the shock name, i.e. **pars.std_shock_gdp = 0.8;**
- If we do not specify the value of the standard deviation, then IRIS will set it to 1, *without a warning message*

Observed variables

- Values for the observed variables must be provided as time series, collected in a database
- Most often we load data from a **csv** file, and then transform this data into model variables
- An example
 - In the model, we use the variable **dl_cpi** as annualized quarterly change of CPI, expressed as percentage
 - In the **csv** file, we have monthly data for the level of CPI, named **CPI**, which we load into a database **dm**
 - Then we must write

```
dobs.dl_cpi = 4*100*diff(convert(log(dm.CPI),'q'));
```

Reading the model into MATLAB

- If we declared the model in a file that is called `model_file.mod`, then we load it into MATLAB by `m = model('model_file.mod');`
- By default IRIS assumes that a model is nonlinear; if it is linear, then we must say `m = model('model_file.mod','linear',true);`
- Next, we have to assign values to the parameters. Assuming these are in the database called `pars`, we say `m = assign(m,pars);`
- We then have to calculate the steady-state, and linearize the model:

```
m = sstate(m);  
m = solve(m);
```

- *Caution!* For linear models, these line must be swapped:

```
m = solve(m);  
m = sstate(m);
```

Modeling in IRIS

Models and their applications

Defining models

Filtering

Simulating/forecasting

Nonlinear models



Filtering

- Use the **filter** function of IRIS to estimate values of the unobserved variables, using the Kalman-smoother
- The syntax is **filter(m,dobs,rng)**, where **dobs** is the database of the measurement variables, and **rng** is the range on which we'd like to filter
- There are many outputs, but in most cases we are only interested in the second, which contains the filtered/smoothed values, so we write **[~,dsm] = filter(m,dobs,rng);**
- The output **dsm** has three fields: the means estimates, the std-s of the estimation errors, and the covariances of the estimation error

dsm =

```
mean: [1x1 struct]
std: [1x1 struct]
mse: [68x5x5 tseries]
```

- For example, the estimated output gap is **dsm.mean.gdp_gap**

Modeling in IRIS

Models and their applications

Defining models

Filtering

Simulating/forecasting

Nonlinear models



Simulation

- There are two command that are very similar
- Both `simulate` and `jforecast` calculate forecasts
- But `jforecast` returns also forecast errors, and it handles conditional forecast differently
 - `j` is for *judgemental*
- We will start with `simulate`
- The syntax is `dsim = simulate(m,dobs,rng);`
- `dobs` is mostly used to get starting values of the transition variables, and `rng` is the simulation/forecast range
- The output will contain only the forecasted values, this can be overwritten, see options in `irisman`

Expert judgement on the shocks

- By default, IRIS assumes that the values of the shocks are 0 on the simulation range
- But this can easily be overwritten by providing explicit nonzero values for the shocks in `dobs`
- Example
 - Suppose `shock_gap` is the name of a transitional shock in the model
 - Let the simulation range be `rng = qq(2015,4):qq(2017,4)`
 - If we know that in 2017Q1 there will be a 1% gap shock, then we set `dobs.shock_gap = tseries(qq(2017,1),1)`
 - Then call `dsim = simulate(m,dobs,rng);` as before

Expert judgement on the endogenous variables

- But it is more common that we have expert information about the specific value of an endogenous variable, and not of a shock
- The primary examples are exogenous variables, but we might also have opinion about policy interest rates and other near term forecasts (NTF)
- Example: we know that our monetary policy committee will not change interest rates for the next three quarters
- So what we want is to run the simulation, but for the first three quarters somehow "turn off" the equation for the policy rate
- This is done by using *simulation plans*

Simulation plans

This is how to set a simulation plan to accomplish the task:

```
rng          = qq(2015,4):qq(2017,4);  
ntf_rng      = qq(2015,4):qq(2016,2);  
pln          = plan(m,rng);  
pln          = exogenize(pln,'i_nom',ntf_rng);  
pln          = endogenize(pln,'shock_monpol',ntf_rng);  
dobs.i_nom   = tseries(ntf_rng,1.5);  
dsim         = simulate(m,dobs,rng,'plan',pln);
```

Obviously, in a simulation plan we can set values for more than one variable, and for any range

Conditional forecasts with jforecast

- To introduce expert judgement with **simulate**, we had to specify which shock explains out forecast
 - we can have expert information on the near-term value of inflation, and also about whether it results from a supply shock or a demand shock
- But it is also possible that we want the model to chose the contribution of the shocks in explaining the forecast value
- For this, we can use the **condition** function of simulation plans as follows

Using condition

This is how:

```
rng                = qq(2015,4):qq(2017,4);  
ntf_rng            = qq(2015,4):qq(2016,2);  
pln                = plan(m,rng);  
pln                = condition(pln,'i_nom',ntf_rng);  
dobs.i_nom         = tseries(ntf_rng,1.5);  
dsim               = jforecast(m,dobs,rng,'plan',pln);
```

- IRIS will automatically set the values of *all transitional shocks* to reproduce the desired value of 1.5
- The contribution of each shock will be proportional to its standard deviation
- But this can only be done with linear models, so IRIS will always use the linear approximation for **jforecast**

Modeling in IRIS

Models and their applications

Defining models

Filtering

Simulating/forecasting

Nonlinear models



Specifying nonlinear models

- We specify the nonlinear equation of a model the same way as any other equation
 - But: if we want IRIS to work optimally, then it is advisable to mark the non-linear equations by using the `=#` sign instead of the usual `=` sign
- All the basic commands (i.e. `model`, `sstate`, `solve`) are also the same
 - But: we must calculate the steady-state before we can solve the model
- The `filter` command also works exactly the same way as before
 - This is because for filtering we must always use a linear model, so IRIS will automatically use the linearized version of a nonlinear model
- And the same is true for `jforecast`-ing

Simulating nonlinear models

- The `simulate` command also works as before:
`dsim = simulate(m,dobs,rng);`
- But now there is a possibility to select a simulation method
 - `dsim = simulate(m,dobs,rng,'method','firstorder');` will use the linear (i.e. first order) approximation; this is the default setting
 - `dsim = simulate(m,dobs,rng,'method','global');` will totally ignore the linear approximation
 - `dsim = simulate(m,dobs,rng,'method','selective');` will use the linear approximation for the equations not marked as nonlinear with the `=#` sign
- In general, the `'selective'` method is the most accurate/efficient

Accuracy of the linear approximation

- The `lhsmrhs` function can be used to check the accuracy of the linear approximation (left-hand-side minus right-hand-side)
- `dif = lhsmrhs(m,db,rng);` will return in `dif` the left-hand-side minus right-hand-side differences, when using the data in the database `db`

- For example:

```
[~,dsm] = filter(m,dobs,rng);  
dif = lhsmrhs(m,dsm.mean,rng);
```

will return the approximation error using the smoothed database

- `dif` will be a numeric matrix where each row corresponds to an equation of the model, with observation equations coming first, followed by transition equations, in the order in which they appear in the model file



Time series models

Time series models

VAR models

ARMA models



Specifying VAR models

- To specify a VAR model object `V` we have to give the names of its variables: `V = VAR({'dl_gdp','dl_cpi','i_nom'});`
- We specify the order of the VAR (i.e. the number of lags) when we estimate the VAR: `V = estimate(V,db,rng,'order',lag);`
- Then we can look at the estimation results by using the `get` function
 - `get(V,'A*')` returns the coefficient matrices as a 3-dimensional array
 - `get(V,'omega')` returns the residual covariance matrix
 - `get(V,'const')` returns the vector of constants
 - `get(V,'SBC')` returns the Schwartz/Bayesian information criteria

Forecasting with VAR-s

- It is straightforward:

```
fc = forecast(V,db,rng(end+1):rng(end+12));
```

will produce forecasts for 12 periods after the end of the estimation range

- It is again possible to introduce judgement into the forecast:

```
cond.res_d1_gdp = tseries(rng(end)+1,0.1);  
cond.d1_cpi      = tseries(rng(end)+1,0);  
fc              = forecast(V,dobs,fcast_rng,cond);
```

- The output contains the mean and the error of the forecast
- The effect of endogenous tunes will be distributed between all the residuals

Time series models

VAR models

ARMA models



Specifying ARMA models I

- ARMA models must be specified in model files
- And so forecasting/simulation is the same as with other models
- Only caveat: in an IRIS model we cannot have lags of transition shocks
- So we have to introduce an auxiliary transition variable

An ARMA(3,2) model

The is how to specify an ARMA(3,2) model

```
!transition_variables  
y, eps_aux
```

```
!transition_shocks  
eps
```

```
!parameters  
c, phi1, phi2, phi3, theta1, theta2
```

```
!transition_equations
```

```
y = c + phi1*y{-1} + phi2*y{-2} + phi3*y{-3} + ...  
    eps_aux + theta1*eps_aux{-1} + theta2*eps_aux{-2};  
eps_aux = eps;
```

```
!measurement_variables  
y_obs
```

```
!measurement_equations  
y_obs = y;
```


Estimation of ARMA models

And this how to estimate it (assuming **db** contains observations for **y_obs**)

```
m = model('arma32.mod','linear',true);

p.c      = 0; % Here we set initial values
p.phi1   = 0;
p.phi2   = 0;
p.phi3   = 0;
p.theta1 = 0;
p.theta2 = 0;

m = assign(m,p);
m = solve(m);

rng = dbrange(db);

init.c    = {nan,-10,10}; % Here we set upper and lower bounds
init.phi1 = {nan,-1,1}; % the NaN-s mean to use the initial values from p
init.phi2 = {nan,-1,1};
init.phi3 = {nan,-1,1};
init.theta1 = {nan,-1,1};
init.theta2 = {nan,-1,1};

p_est = estimate(m,db,rng,init,'nosolution','penalty');
m = assign(m,p_est);
m = solve(m);
```



Disclaimer

This document and the information contained within are intended only for the person or entity to whom they are addressed and may contain confidential, proprietary and/or privileged information. Any unauthorized disclosure or distribution to persons or entities other than the intended recipient is prohibited. The information and opinions contained herein do not necessarily reflect OGREsearch's views and OGREsearch does not warrant the correctness thereof. This document is for informational purposes only, it is not intended as a solicitation or offer to enter into any transaction or business relationship, and should not be taken as advice or a recommendation to buy or sell any financial instruments or securities. OGREsearch does not accept liability for any damage caused by reliance on the contents herein.